

Documentación del proyecto

Se debe realizar la siguiente documentación por cada proyecto.

Fase 1. Contextualización

- ¿Qué se debe hacer?
- ¿Qué se necesita?
- ¿Qué elementos se identifican

Se debe implementar una “cola de impresión” para una impresora. Esta debe utilizar como base la estructura de datos de una queue o cola, de manera que permita encolar nuevos elementos para imprimir, y al presionar imprimir debe eliminar el elemento más antiguo que haya (el que esté primero en la cola).

Para llevar esto a cabo, programaremos una app fullstack que funcione por medio de un Front-End con React y una API con FastAPI para

Fase 2. Definición de los requerimientos funcionales

Nombre	R1 - Subir archivos a la cola
Descripción	Ser capaz de añadir archivos a la cola de impresión para que puedan ser visualizados.
Entrada	Un archivo cualquiera que se pueda imprimir
Salidas	El archivo debe verse reflejado en la cola de impresión

Nombre	R1 - Imprimir archivos
Descripción	Al presionar el botón de imprimir un archivo, el archivo que se encuentre en la primera posición de la cola debe eliminarse (imprimirse)
Entrada	Presionar el botón de imprimir
Salidas	El archivo en la primera posición debe desaparecer de la cola

Fase 3. Diseño

Para este programa, se decidió implementar la Queue en lugar de como una clase, como una API que re-crea las funciones de esta. De esta manera, la API cuenta con end-points que funcionan de la misma forma que una clase Queue, pero interactuando con un archivo .json que almacena los nombres de los archivos que están subidos de forma persistente, y subiendo los archivos a un directorio en el servidor.

Nombre de la API (module) main.py
FUNCIONES: read_queue() : dict write_queue() : None uploadFile() : dict getFiles(): list[str] printFile() : list
ENDPOINTS: POST /uploadfile → uploadFile() (Enqueue) POST /getfiles → getFiles() (Print queue) POST /printfile → printFile() (Dequeue)

Fase 4. Codificación

Para hacer funcionar este programa, se utilizaron:

Para el front-end: El framework React de Javascript, con el que se contruyó el

aspecto visual para reflejar toda la lógica y que esta corra en el navegador.

Para el back-end: la librería FastAPI de Python, para construir APIs.

Utilizando este Stack, se decidió la arquitectura:

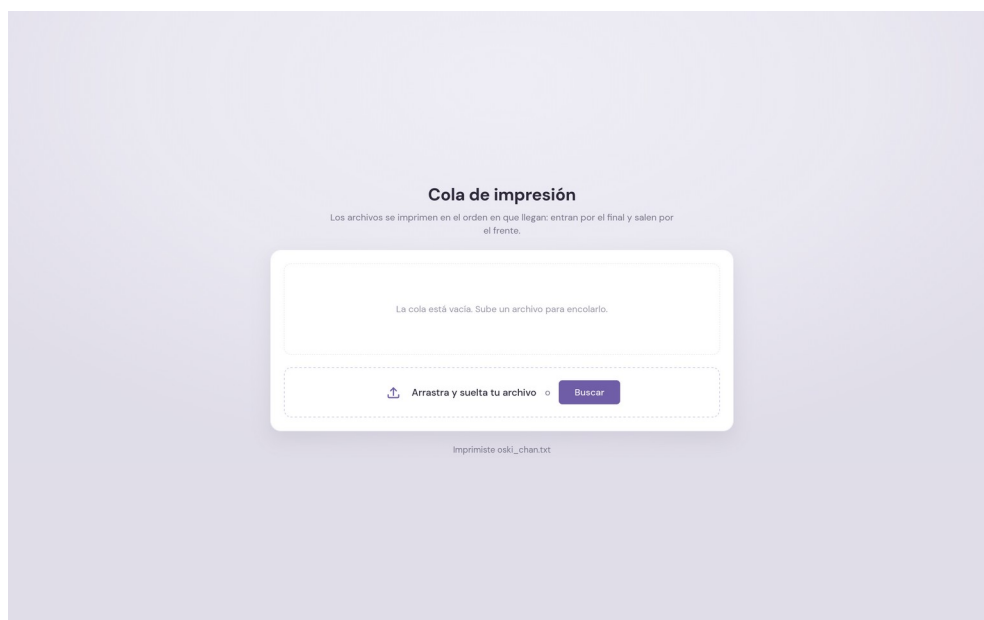
La API debe estar corriendo en un servidor, y tiene una carpeta “files” en la que almacena los archivos que recibe. Esta tiene endpoints que recrean la funcionalidad de una cola común, pero que interactúan con un archivo .json el cuál tiene una lista de todos nombres de los archivos de la carpeta files. La única manera de interactuar con esta carpeta es por medio de los endpoints de la API, que lo actualizan cada vez, por lo que, esta lista siempre se mantiene actualizada.

La API ofrece la capacidad de leer los archivos de la cola, imprimir (quitar el archivo más antiguo), y añadir archivos a la cola subiéndolos.

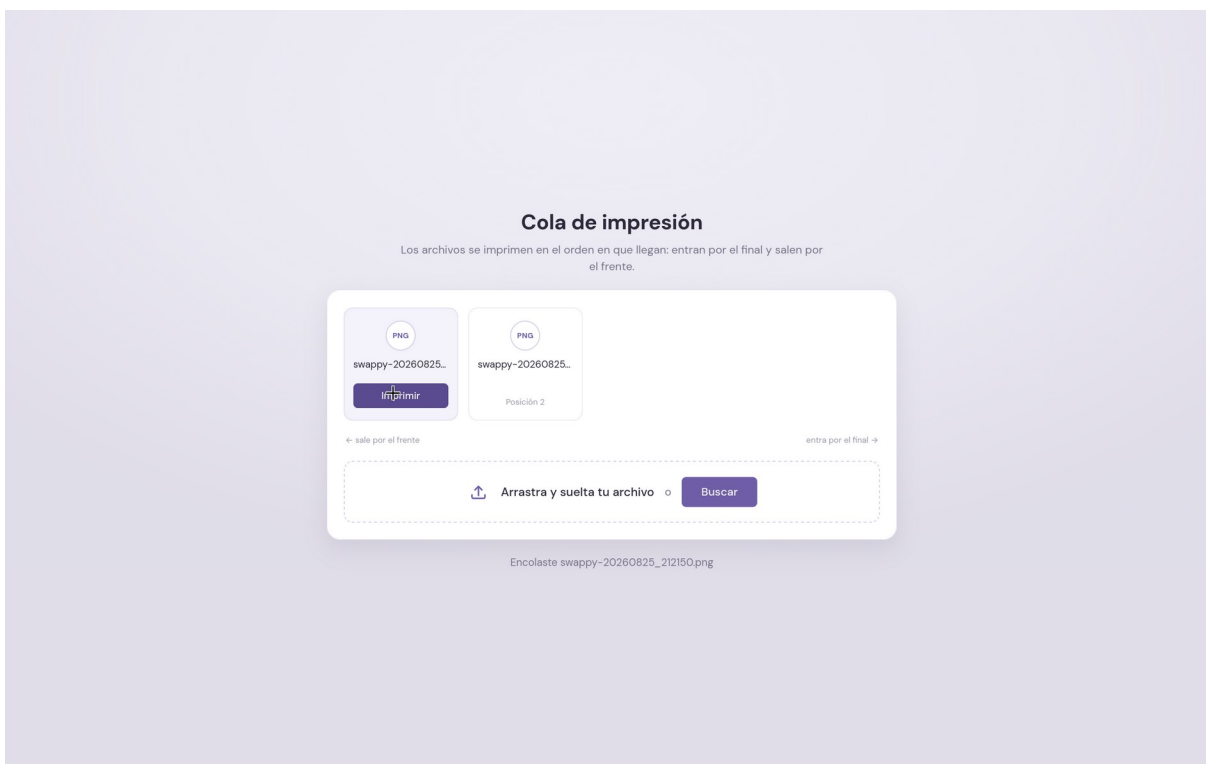
El front-end fue programado de manera que tenga una interfaz visual que, utilizando JavaScript, permita con los botones de la interfaz interactuar con esta API y subir archivos al servidor para añadirlos a la cola. De esta manera, por medio del endpoint para leer los archivos en la cola, muestra todo lo que se encuentra encolado al cliente, y, con la app desplegada correctamente en un servidor, mantiene un estado global para cualquier computadora que entre.

Fase 5. Pruebas.

Cola vacía:



Cola después de subir archivos arrastrando o buscando con el botón:



Cola después de imprimir un archivo:

Cola de impresión

Los archivos se imprimen en el orden en que llegan: entran por el final y salen por el frente.

PNG

swappy-20260825...

Imprimir

← sale por el frente

entra por el final →

Arrastra y suelta tu archivo

Buscar

Imprimiste swappy-20260825_211856.png